

Yen Compiler

An end-to-end technical deep dive

RAYEN AHMED BRAHIM

Contents

1	Yen Compiler	2
1.1	The Lexer (Scanner)	2
1.1.1	The Lexer class	3
1.2	The Parser	4
1.2.1	Formal Grammar	4
1.2.2	Why we use the term “Tree” in AST	7
1.2.3	Synchronization	7
1.3	The Visitor pattern	8
1.4	Resolver	9
1.5	Type checks	11
1.6	Bytecode compiler/VM	11
1.6.1	The VM	12
1.6.2	Binary expressions	13
1.6.3	Variable declarations	14
1.6.4	Variable expressions	14
1.6.5	Logical expressions	14
1.6.6	While loop	15
1.6.7	Function calls	16
	Closing Note	18

Preface

This project was not built to ship a production language, it was built with the intention of learning how compilers work and what a programming language does under the hood. It's also practice at engineering a real system end to end, not just reading about one.

This document walks through the compiler pipeline in the order source code actually flows through it: the Lexer turns raw text into tokens, the Parser turns tokens into an Abstract Syntax Tree (AST), the Resolver figures out what each variable reference actually refers to, the Type Checker verifies type correctness, and the Bytecode Compiler turns the validated AST into a compact set of instructions that the VM executes.

1

Yen Compiler

Yen is a statically typed, bytecode compiled scripting programming language that runs on a custom VM, implemented in java.

1.1 The Lexer (Scanner)

Yen is a scripting language, that means it runs directly from source, it doesn't produce a separate executable. But this doesn't mean what we're building is an interpreter it is a compiler that executes the program directly after compiling instead of outputting an executable that need to be run deliberately after compilation.

usage: `yen [file.yen]`

The starting point for the compiler is the main function found in the Compiler class. It reads the argument array provided by the user, checks if the length is 1 since we compile one file at a time, then it calls the `runFile()` passing the source file name as an argument. The file name will be converted into a system path object and the target file will be fetched. The content of the target file will be read byte by byte and will be stored inside a byte array/buffer as raw bytes. The raw bytes buffer will then be passed to the `run()` method as a long

string of characters “the source string”. Inside the run() method the source string will be used to construct a new Lexer (Scanner) object.

1.1.1 The Lexer class

An Object of the Lexer class has the following attributes:

- String source
- List tokens –the final list that contains the tokens.
- A start pointer that points to the start of the current token. And a current pointer that points to the current character of the current token being processed. And a line indicator pointing to the current line.

The scanTokens() method will be invoked returning – at the end an array/list containing every token scanned from the source.

Entering the loop inside scanTokens() for the first time start = 0; current = 0; we’ll keep traversing the source string character by character and adding tokens to the tokens array as long as we aren’t at the end. The scanToken() method (called on each iteration of that loop) will see the current one or more characters (if necessary) and emits a token accordingly. For example if the current char is ‘(’ it appends a LEFT_PAREN token to the end of the token list. If it is a variable name it will read it character by character and it will append an IDENTIFIER token that will internally contains the variable name as a string and its line number. White spaces are skipped and new lines are skipped after incrementing the line counter. Comments are also skipped by skipping characters until reaching the end of line.

1.2 The Parser

The parser will take the linear stream generated by the lexer and create abstract syntax trees ASTs according to the rules of the formal grammar (precedence, associativity..) creating a nested structure that represents the expression's interpretation using the language formal grammar rules. It is at this stage where the plain stream of tokens gain its meaning and structure. This is also where syntax errors are caught.

1.2.1 Formal Grammar

Yen's formal grammar rules, in Backus-Naur Form (BNF):

```
expression -> assignment ;
assignment -> IDENTIFIER "=" assignment
            | logic_or ;
logic_or    -> logic_and ( "or" logic_and )* ;
logic_and   -> equality ( "and" equality )* ;
equality    -> comparison ( ( "!=" | "==" ) comparison )* ;
comparison  -> term ( ( ">" | ">=" | "<" | "<=" ) term )* ;
term        -> factor ( ( "-" | "+" ) factor )* ;
factor      -> unary ( ( "*" | "/" ) unary )* ;
unary       -> ( "!" | "-" ) unary
            | primary ;
primary     -> NUMBER | STRING | "true" | "false"
            | "(" expression ")"
            | IDENTIFIER ;

program     -> declaration* ;
declaration -> varDecl | funcDecl | statement ;
statement   -> exprStmt | ifStmt | printStmt | whileStmt
            | blockStmt | returnStmt | forStmt ;
```

* means zero or more occurrences, | separates alternatives, and () groups a sequence of symbols together. Each rule reads top to bottom in decreasing precedence – **assignment** binds loosest, **primary** binds tightest – which is exactly the order the recursive descent parser walks through below.

Our parser uses recursive descent to mirror the nested nature of the grammar. Each rule in the grammar is a method that contributes to the construction of an AST node.

After obtaining the flat array of tokens from the lexer, we create a new parser object and start parsing the tokens resulting in a series of statements, each of these statements is an AST by itself that contains nested nodes.

A program is a sequence of statements and a unique type of statements is declarations, so we start the parsing by checking if we have a declaration. Declarations start with tokens that indicate a type “INT, DOUBLE, STRING, BOOL, VOID”, since our language is statically typed. If we have a declaration we store the type token and the identifier token –which contains the name then we look for a left parenthesis to find out if it is a function declaration or a simple variable declaration. If it is the latter we look if the next token is an “EQUAL” token to find out if we have an initializer, if it is the case we parse the initializer expression otherwise it will be set to null, then we make sure that the statement ends with a semicolon if not, a syntax error will be reported. Then we return a new varStmt node.

If we’re dealing with a function declaration, we parse the function parameters then parse the function body as a list of statements and return a new FunctionStmt node.

If we're not dealing with a declaration we look for one of the simple statements according to the current token "FOR, IF, PRINT, RETURN, WHILE or a block". For example if the current token is a "FOR" then we have a for statement, the dispatch will end up calling `forStatement()` method. The special thing about for statements is that they don't exist as a mechanism outside being a syntactic sugar to make writing loops cleaner and easier in some cases, so for loops get desugared into while loops as follows: First we store the FOR keyword to use it to access line numbers in later stages since tokens have a line field. We consume the left parenthesis then we parse the loop counter initializer if it exists, parse the condition if it exists, if it doesn't it'll be assumed "true". And then we parse the increment expression if it existed. After that we move to the body, we parse the body, add the increment expression to the end of the body, then we create a `whileStmt` node using the condition and the parsed body as arguments to the `whileStmt` constructor, and finally we add the initializer before the while statement and return this sequence of statements.

Other statements have similar structure and are parsed in the following way:

- we match the statement keyword.
- we get dispatched to the correct method dealing with that specific statement.
- we consume the expected next token if it doesn't exist we report a syntax error.
- we collect the expected data and store it in local variables temporarily "conditions, elseBranch, a value, initializer..."
- we construct the corresponding AST node.

If the statement we're parsing is not a statement then in this case we'll do a recursive descent that mirrors the language grammar rules (Backus naur grammar rules) starting from the top –lower precedence and going down the grammar rules until we arrive to the primary tokens which are literals, identifiers and parenthesised expressions. Literals and identifiers get stored as literal expression nodes and variable ex-

pression nodes respectively, constituting the leaves of the ASTs, while parenthesised expressions get stored as grouping expression nodes wrapping whatever expression was inside the parentheses.

1.2.2 Why we use the term “Tree” in AST

In the Expr subclasses – Binary, Unary, Grouping, Literal – these subclasses don’t hold raw values directly, they hold fields. Binary, for example, holds fields like Expr left and Expr right. Those fields are themselves Expr objects, which could be another Binary, Unary, or Literal Expr. So when you parse `1 + 2 * 3` you don’t get a flat list, you get:

```
Binary (+)
|-- Left:  Literal (1)
'-- Right: Binary (*)
           |-- Left:  Literal (2)
           '-- Right: Literal (3)
```

This nesting is the tree – each class instance is a node.

1.2.3 Synchronization

When an error occurs, we throw an error and catch it at the first calling method “parse()”. At this point the recursive call frame where the error occurred is popped. The error gets reported but the parser doesn’t stop and compilation doesn’t halt yet, the parser keeps going in order to find other eventual errors. For this we need to synchronize the parser i.e. we need to skip tokens that are related to the flawed line or statement and were going to get processed in the recursive calls of the popped stack frame. To do that we keep skipping tokens until we reach something that indicates the end of the current statement containing the error – like a semicolon or the start of a new statement like a type, for statement, if statement.

1.3 The Visitor pattern

The object-oriented paradigm relies on classes to instantiate objects, each class has its instances and the class defines behaviour using methods. But the problem is – let’s say you have different classes and behaviours like `walk()`, `fly()`, each class has its own implementation of `walk()` and `fly()`, now let’s say we want to define a new behaviour `dive()`, in order to do so you have to walk through every single class and add the corresponding implementation of `dive()` which is not a very fun and efficient thing to do when you have hundreds of classes. On the other hand, functional paradigms handle this problem sort of naturally, you’ll have one `walk()` function one `fly()` and one `dive()`. Each function has a way to pattern match which type is being processed and execute the correct piece of code, but imagine we want to add a new type, this will require walking through every function and add the case that processes this new type. Again, this is too much work when you have hundreds of functions.

The visitor pattern aims to approximate the convenience of the functional paradigm in an object oriented language. It works by having an interface that has one generic method for each type and an `accept()` method for each class that route back to the correct implementation that handles that class type.

Let’s say you’re processing different types, and since the types are different you don’t know which type you are currently treating, for example you want to print but each type is printed in a different way, so what you do is pass the `print()` method as a visitor to the `accept()` method, and here the `accept` method specific to that type is the one getting invoked. So the `accept()` method for that type will execute, for example: we’re processing a binary expr, so its `accept` method looks like this:

```
@Override
<R> R accept(Visitor<R> visitor) {
    return visitor.visitBinaryExpr(this);
}
```

In the print class we have:

```
private void print(Expr expr) {
    expr.accept(this);
}
```

Here the `print()` method is getting passed to the expression's `accept` method, then the `accept` method will call the `visit` method that handles its type inside the print class passing itself to get handled.

This is called double dispatch. Unlike single dispatch –normal method calls which picks behaviour based on one type “the runtime type”, the visitor gets you dispatch on two types simultaneously – the type of the AST node “`accept(this)` of that node type is the one invoked” and the type of the visitor (operation) “`visitor.visitX()`”

1.4 Resolver

After parsing is finished, we're left with a list of statements which are ASTs of various types and depths. The next step is binding/resolving, which kind of gives a fixed identity to each declared variable so when it is encountered later we know which variable is getting referred to. It is the way we tell the computer that `x` refers to `x` declared as `int x = 1` and not to `int y = 0`. We are creating an association between a name and a storage location identified by its slot index, which brings us to another important utility we get from this stage which is indexing, it will enable us to access variables in $O(1)$ at runtime. We are giving each variable a slot index according to its relative position in the code. Each time we declare a variable we associate it with a slot index, if it is a local variable that slot index is relative to the current scope we're in.

For example we have declared two local variables so the next slot index is 2 (starting from 0), we enter a new block, but before that we save the `currentSlotIndex` which is 2, then inside the block let's say we declared 3 vars so the `nextSlotIndex` becomes 5, after that we exit that block, we revert back to the saved `nextSlotIndex` which is 2 because the previous 3 variables are no longer in scope. This enables us to reuse slot indices that were occupied in scopes deeper than that current one, so two variables can have the same slot index if they're in two different scopes. Slot allocation emulates how the virtual machine will operate and use local variables.

For the mechanisms, we have a hash map `<name, symbols>` for globals, each time a global is declared we construct a symbol containing the name, type, Kind (variable, function or a parameter), scope depth and the variable's slot index. Every next time the variable is used we look it up on the hashmap and assign the same symbol to it, so all appearances of the same variable will carry the same symbol and the same slot index. We do the same thing for locals except that we don't have a single hashmap but we have a stack of hashmaps, one map per scope, this enables shadowing so even if two variables have the same name we'll reference the one closest, this enables us to use the same variable name multiple times in different scopes.

When we enter a new scope we push a map onto the stack, each variable declared inside the current scope is associated with a symbol on the map found on the top of the stack. Next time when we encountered that variable we look for it from the top of the stack downward i.e. from the innermost scope outwards until we reach global scope.

1.5 Type checks

After resolving, our AST nodes are annotated with symbols containing slot indices and types. The job of the type checker is to verify type compatibility. You can't assign a double to a variable of type string, you can't perform addition between an integer and a boolean, you can't return a value from a void function either.

Note that Yen has int to double widening, meaning you can assign an integer to a double but that integer will be transformed to a double object later at the VM stage. Yen also has built in string concatenation so if both operands type is STRING and operator is '+' then string concatenation will be performed. Other expressions/statements are mechanical.

1.6 Bytecode compiler/VM

After the type checker pass, our ASTs are fully decorated and full of information reflecting the intended structure by the source code.

We'll traverse the ASTs one last time to emit bytecode, a set of instructions to be executed by our VM. Each instruction has an opcode, like OP_ADD, OP_CALL, OP_JUMP. After this stage we won't need the ASTs anymore and the whole program will be represented by a stream of instructions – no variable names, no loops just raw byte instructions, jumps and slot indices to obtain maximum performance. A raw byte dynamic array `code[]` will store the program instructions. This array is found on a Chunk object, each chunk has its `code[]` array, `lines[]` array and a constant pool where constants and literals will be stored. They'll be referenced by a byte sized index that points to their location in the constants pool.

We start by passing the program statements list to the `compile()` method of the BytecodeCompiler object. It will create a new chunk, this is the main program's chunk. Each function has its own chunk, the main program is also treated as a function at the VM stage. As the

program starts a new call frame is pushed, the main program's chunk is loaded and the program starts running. When a function is called, a new call frame is pushed and the function's own chunk found on the constant pool at a certain index is loaded, we'll get to that later.

Then we iterate through the statements compiling them one by one, they will be dispatched to their corresponding method using the visitor pattern.

1.6.1 The VM

After compiling the program to Bytecode the virtual machine is responsible for interpreting and executing instructions. The VM will manipulate a callFrame stack pushing call frames for the main program and function calls and popping them after returning. The different objects the VM will use to perform operations and local variables will both live on a stack of objects, and global vars will live on an array.

Turning on the VM will push a new call frame onto the callFrame stack. Each call frame has its own chunk, instruction pointer and an integer index indicating the start of the objects relative to the callFrame, called frameBase.

slotIndex is allocated relative to the current scope, it expands and shrinks, and a slotIndex can be reused by two different variables living in different scopes – this means the slot location isn't an absolute stack position, and every function's locals start counting from slot 0 regardless of how deep the call stack is. So at runtime we can't index the stack directly by slot index; we need frameBase as a per-call base index, treating the start of the current call frame as the base and the slotIndex as an offset from it: local variable *n* inside the current function sits at `stack[frameBase + n]`.

Since slot indices and other operands are stored as single bytes, and Java's byte is signed (-128 to 127), any operand value above 127 gets read back as a negative number due to sign extension – for example a slotIndex of 200 gets stored as -56, and reading it back naively as an

int still gives -56. Masking the byte with `& 0xFF` fixes this: when Java promotes that negative byte to an int, the sign bit gets extended into all the upper 24 bits, but ANDing with `0xFF` clears those upper bits and keeps only the original 8 – so even though the byte’s own sign bit was 1, the resulting int’s sign bit (bit 31) comes out 0, and we correctly get 200 back.

After that we enter the `run()` method which is a big dispatch loop that handles every opcode of the current `callFrame`’s chunk as long as the `callFrame` stack is not empty. We start reading the instructions byte by byte executing the corresponding piece of code.

With the compiler and VM’s general shape in place, we’ll cover some interesting node types starting with the simplest, walking through how each one gets compiled and then executed:

1.6.2 Binary expressions

We compile the left side then check if it is an Integer with the expression’s type being double, we emit a `OP_INT_TO_DOUBLE` instruction to apply the widening. We do the same thing for the right side then we select the correct binary opcode to emit according to the expression’s type and the operator type, including string concatenation and comparison.

Comparison opcodes (`OP_LESS`, `OP_GREATER`, etc.) aren’t type-specialized like the arithmetic ones though – the result of a comparison is always a bool regardless of whether the operands were ints or doubles, and converting both operands to double before comparing gives the correct result either way, so a single generic comparison opcode is enough, no need to double the opcode count the way arithmetic needed to.

At runtime: we pop both operands from the top of the stack in the correct order, execute the desired operation and push the result back on the stack.

1.6.3 Variable declarations

First we check if the declared variable has an explicit initializer, if yes we compile it otherwise we give it the correct default zero value for its type “0 for int, 0.0 for double, false for bool and” “ empty string for strings”. Either case will push the initializer value on the top of the stack, then we emit either `DEFINE_GLOBAL slotIndex` to store the initializer value on the correct slot of the globals array since globals are stored on an array unlike locals which are stored directly on the stack.

At runtime: variable declarations are handled by reading the `slotIndex` and storing the top value of the stack on the globals array cell located on that index. In the case of locals we just continue since locals live directly on the stack and will be placed in the correct slot naturally according to the invariant that the stack must always stay clear and balanced.

1.6.4 Variable expressions

When we arrive to a `variableExpr` node we need to get that variable value on the top of the VM stack in order to be able to use it, so we check its symbol’s scope depth to find out if it is a global or local variable, then we emit the correct opcode `OP_GET_GLOBAL/LOCAL` along side with the symbol’s slot index casted as a byte operand.

1.6.5 Logical expressions

To gain performance we can short circuit the logical expressions since having only one side false in a logical and operation will evaluate the whole expression to false and having at least one side true on a logical or operation will evaluate the whole expression to true. So we can skip the rest of the expression if the evaluation result is already determined. But since all we have is a linear stream of instructions we can’t find out how long to skip before compiling the rest of the instruction. To solve this we use jump patching, we’ll emit a jump instruction with two

bytes representing a place holder of the jump distance in big endian, these placeholder bytes will be filled later when the jump distance is determined. Let's walk through a concrete example, starting with the end operation. We have (`leftSide`) and (`rightSide`) we compile `leftSide` if it is true all good we pop the true value from the VM stack and continue to evaluate the right side, but if the `leftSide` is false we don't need to continue evaluating the `rightSide`, so after compiling the `leftSide`, we emit an `OP_JUMP_IF_FALSE byte byte` then we compile `rightSide`, after the size of the `rightSide` is known we calculate the jump distance needed to jump past the `rightSide` and go back to patch the placeholder bytes with the jump distance.

We use the same technique to handle logical or but with a double jump instead of using a `OP_JUMP_IF_TRUE` opcode.

At runtime: to handle control flow for jumps we just increment the instruction pointer by the distance specified on the instruction, and for the loop instruction we decrement it.

1.6.6 While loop

The special thing about while loops is that we need to repeat the same part of instructions many times and to do that we need to save the location where the loop start to keep jumping back, then we compile the condition if it is false we jump past the loop by patching the jump distance after compiling the body. If the condition is true we pop the true literal from the VM stack, compile the body, emit an `OP_LOOP` instruction with a distance to jump backwards and land at the loop start to repeat the set of instructions.

Note that jump instructions increment the instruction pointer by the jump distance and the loop instruction decrements it.

1.6.7 Function calls

Since functions live in their own callFrame they need their own chunk, so to deal with a function declaration we save the current chunk, create a new chunk, compile the function's body inside this new chunk and emit an unconditional safety `OP_RETURN_VOID` to handle functions that don't have an explicit return – we don't statically verify that every path through a function's body ends in a return, so if a non-void function falls off the end without one, this safety `OP_RETURN_VOID` is what stops execution from running past the end of the chunk.

Then we create a `YenFunction` object which is just a wrapper to avoid storing a bare chunk, restore the current chunk back to the enclosing one we saved earlier, and store the `YenFunction` on that enclosing chunk's constants pool, since the function itself becomes a constant/global of the chunk that declares it, not of its own body's chunk. So compiling a function declaration comes down to: get the function's chunk of bytecode, wrap it as a `YenFunction` object, put it in the constant pool, emit `OP_CONSTANT` with that index so the `YenFunction` gets pushed onto the stack, then `OP_DEFINE_GLOBAL` so the global variable at that slot holds the `YenFunction`. Next time that function name is used, `OP_GET_GLOBAL` pushes that same `YenFunction` back onto the stack – which is exactly what `OP_CALL` will peek to find the chunk to jump into.

When the function is called we first compile the callee so the function value itself gets pushed on the stack, then we compile the arguments so they get pushed right above it, then we emit an `OP_CALL` instruction with the number of arguments as operand so at the VM stage we know where the frameBase starts i.e. the first argument which is the first local of the function (`stackTop - argc`), with the function value itself sitting one slot below that. And after the function is called, compiling the return statement comes down just to compile the return value if it exists and emit the `OP_RETURN_VOID/VALUE` instruction.

At runtime: when we land on the case of a function call we read the argument count since arguments are ready and already pushed on the stack, we perform a bitwise AND to the read value with 0xFF to prevent sign extension, then set the frameBase to point to the first argument. The new call frame gets pushed with the function's own chunk, ip and frame base then it gets executed. To return from a call we just revert the top of the stack to where it was before the call. After that we push the return value on the stack if the function returns a value and pop the function's callFrame.

If the popped call frame is the main program's frame then we'll exit the dispatch loop and the program is terminated.

Closing Note

This walkthrough ends here, with the main program's call frame being popped and the VM terminating at startup's `OP_RETURN_VOID` – from raw characters to bytecode running on a stack machine, that's the compiler end to end.

Yen stays deliberately small: no closures, no first-class functions, no classes, no OOP, no garbage collector of its own, nothing beyond `int/double/ bool/string/void`. This wasn't built to be a production language, it was built to learn how compilers work and get some real practice engineering a system end to end. Each cut is what made it possible to get a complete pipeline – lexer, parser, resolver, type checker, bytecode compiler, VM – working within the time available, while still making real compiler decisions at every stage.

If you're reading this next to the source, hopefully by now the two make sense together.